

CSED211: Cache lab Report

컴퓨터공학과 20240940 장해영

1. Overview

해당 과제는 두 개의 파트로 나뉜다. 파트A는 캐시의 작동방식을 바탕으로 캐시 시뮬레이터를 만드는 것이고, 파트B는 matrix를 transpose하는 코드를 최적화하는 것이다.

2. Part A: Writing a Cache Simulator

A. 캐시 구조 설계

캐시는 여러 개의 set으로 구성된다. 그리고 set은 line으로 구성된다.



캐시는 다음과 같은 구조로 되어있다. v는 valid bit로 0과 1을 이용하여 유효성을 나타낸다. 다음으로 tag는 캐시에 있는 값이 어느 메모리 위치에 있는 지를 나타내는 bits이다. 그리고 나머지는 데이터를 나타낸다.

```
#include "cachelab.h"
#include <getopt.h>
#include <stdlib.h>
#include <unistd.h>
#include <stdio.h>

// define struct
typedef struct{
    int valid;
    unsigned long long tag;
    int lru;
} line_type;

typedef struct{
    line_type* lines;
} set_type;

typedef struct{
    set_type* sets;

    int s; // number of set index
    int E; // number of lines
    int b; // number of block bit
} cache_type;
```

우선 다음과 같이 헤더파일을 추가하였다.

위와 같은 구조를 살펴보았을 때 우선 line에 대한 struct를 만들어야겠다는 생각을 할 수 있다. 이 line_type struct에는 valid, tag, 그리고 LRU정책을 만족하기 위한 LRU를 추가하였다. 그리고 line이 모여있는 set_type도 lines의 배열로 만들었다. 마지막으로 set들이 모여 최종적으로 cache가 되므로 sets배열과, cache크기를 결정하는 s, E, b를 포함한 struct인 cache_type까지 총 3가지의 구조체를 만들었다.

B. Global variable, 함수 선언

```
// global variable
int global_lru = 0;
int hit_count = 0;
int miss_count = 0;
int eviction_count = 0;

// function prototype
void access_cache(cache_type* cache, unsigned long long int address);
```

다음으로 global variable들을 선언하였다. 우선 lru정책을 이용하기 위해 프로그램을 처음 시작했을 때부터 흐른 시간을 측정하는 global_lru, 추후 printSummary에 이용할 3가지 count변수를 global variable로 지정하였다.

C. Main 함수

Main 함수는 크게 다음과 같은 5가지 파트로 나눌 수 있다.

① 변수 선언 ② 옵션 입력 ③ 캐시 초기화 ④ 시뮬레이션 진행 ⑤ 마무리 작업

① 변수 선언

```
int main(int argc, char *argv[])
{
    // variables
    cache_type cache = {0};
    int S = 0; // number of sets
    // int B = 0; // number of blocks, not used
    char* trace_file = NULL;
```

캐시에 정보를 저장하기 위해 cache 변수를 새로 정의하였다. 또한, 입력받을 s는 set의 비트수이므로 실제 set의 개수를 계산하기 위한 변수인 S도 정

의하였다. 이 방식은 block과 관련된 변수에도 동일하게 적용되지만, 해당 시뮬레이터에서는 B를 사용하지 않으므로 주석처리하였다. 또한, 추후 시뮬레이션을 진행할 trace파일명을 입력받으므로 trace_file도 정의해주었다.

② 옵션 입력

```
// getopt
int opt;
int verbose = 0;
while ((opt = getopt(argc, argv, "hvs:E:b:t:")) != -1){
    switch(opt){
        case 'h':
            printf("Usage: ./csim-ref [-hv] -s <s> -E <E> -b <b> -t <tracefile>\n");
            break;
        case 'v':
            verbose = 1;
            break;
        case 's':
            cache.s = atoi(optarg);
            S = 1 << cache.s; // s^2
            break;
        case 'E':
            cache.E = atoi(optarg);
            break;
        case 'b':
            cache.b = atoi(optarg);
            // B = 1 << cache.b; // b^2
            break;
        case 't':
            trace_file = optarg;
            break;
    }
}
```

사용자로부터 각 옵션에 해당하는 정보를 입력받는다. h와 v 옵션의 경우 따로 입력값이 없으므로 뒤에 콜론(:)을 추가하지 않았고, 나머지 s, E, b, t의 경우 각 옵션에 해당하는 입력값이 필요하기에 콜론을 같이 입력해주었다. h의 경우

사용법을 알려주는 옵션이므로 사용법인 "Usage: ./csim-ref [-hv] -s <s> -E <E> -b -t <tracefile>"를 출력해준다. v의 경우 verbose로, trace정보를 알려주므로 추후 trace파일에서 정보를 읽은 후 출력할 수 있도록 verbose를 1로 바꾸어준다. 다음으로 s, E, b의 경우 각 옵션에 맞는 정보를 각 변수에 string을 int로 바꾸주는 함수인 atoi를 이용하여 저장해준다. 마지막 옵션인 t는 trace파일명을 입력받으므로 이를 trace_file에 그대로 저장해준다.

③ 캐시 초기화

```
// initialize cache
cache.sets = (set_type*) malloc(sizeof(set_type) * S);
// line
for (int i = 0; i < S; i++){
    cache.sets[i].lines = (line_type*) malloc(sizeof(line_type) * cache.E);
    for (int j = 0; j < cache.E; j++){
        cache.sets[i].lines[j].valid = 0;
        cache.sets[i].lines[j].tag = 0;
        cache.sets[i].lines[j].lru = 0;
    }
}
```

2번단계에서 받은 캐시 정보를 바탕으로 캐시를 초기화하는 단계이다. malloc함수를 이용하여 set의 크기와

set의 개수에 해당하는 수를 곱하여 동적으로 메모리를 할당해주었다. 다음으로 line에 해당하는 메모리를 각각 할당해주며 각 라인에 대해 valid, tag, lru값을 0으로 초기화하였다.

④ 시뮬레이션 진행

```
// read trace file
FILE* fp = fopen(trace_file, "r");
char type;
unsigned long long int address;
int size;

while (fscanf(fp, " %c %llx,%d", &type, &address, &size)>0){
    if (verbose) printf(" %c %llx,%d\n", type, address, size);

    switch(type){
        case 'I':
            break;
        case 'L':
            access_cache(&cache, address);
            break;
        case 'S':
            access_cache(&cache, address);
            break;
        case 'M':
            access_cache(&cache, address); // load
            access_cache(&cache, address); // store
            break;
        default:
            break;
    }
}
fclose(fp);
```

trace파일을 읽어들이며 실제 시뮬레이션을 진행하는 단계이다. 우선 fscanf를 이용하여 trace파일에 있는 내용을 한 줄씩 읽어준다. 이 때 address는 긴 주소가 들어올 수도 있기에 타입을 unsigned long long int 로 지정해주었다. 다음으로 위의 verbose옵션이 활성화되었을 때 정보를 출력할 수 있도록 printf를 이용하였다. 이렇게 얻은 값을 이용하여 I, L, S, M 총

네가지 타입에 해당하는 수행을 해야 한다. Writeup 문서에서 I는 무시하라고 했으니 I의 경우는 그대로 넘어가준다. 다음으로 L과 S는 각각 load와 store를 의미하므로 주소에 접근하는 행위가 한 번씩 일어난다. 다만, M의 경우 수정하는 타입이므로 load와 store가 모두 한 번씩 일어나 주소에 접근하는 행위가 두 번 일어난다. 따라서 access_cache함수를 두 번 호출해준다. access_cache함수는 추후 설명할 예정이다.

이 모든 과정을 trace파일의 줄 별로 수행한 다음 파일을 닫아준다.

⑤ 마무리 작업

```
// free memory
for (int i = 0; i < S; i++){
    free(cache.sets[i].lines);
}
free(cache.sets);

printSummary(hit_count, miss_count, eviction_count);
return 0;
}
```

위에서 각 set과 line에 대해 동적으로 메모리를 할당해주었기 때문에 free를 통해 모든 메모리를 해제해주어야 한다. 다음으로

캐시 시뮬레이션을 한 결과를 출력해주는 printSummary함수에 hit_count, miss_count, eviction_count인자를 넣어 결과를 출력해준 뒤 함수를 종료한다.

D. Access_cache 함수

메모리 접근이 발생할 때 호출되는 함수이다. 우선 캐시와 데이터의 주소가 어떻게 연결되는지 살펴보자. 첫 t 개의 비트들은 tag를 나타낸다. 다음 s 개의 비트들은 이 데이터가 어느 set에 위치할 지를 정하는 set index를 나타낸다. 그리고 b 개의 비트들은 block offset으로, data중에서도 어디에 있는 데이터를 읽을 지 정한다.

① 데이터 가공

```
void access_cache(cache_type* cache, unsigned long long int address){
    global_lru++;
    int s = cache->s;
    int b = cache->b;

    // set index
    unsigned long long int mask = (1 << s) - 1;
    unsigned long long int set_index = (address >> b) & mask;
    // tag
    unsigned long long int tag = address >> (s + b);
    // setting cache set
    set_type* set = &cache->sets[set_index];
    int E = cache->E;
```

trace파일에서 얻은 정보는 주소이다. 이 주소를 캐시에 저장할 수 있는 값으로 변환하는 작업이 필요하다. 우선 set index를 알아야 한다. Set index는

주소의 중간 부분에 있는 s 개의 비트들로 알 수 있다. 이 비트를 추출하기 위해 우선 오른쪽에 있는 b 개의 비트를 제거해주어야 한다. 즉, address를 b 만큼 오른쪽으로 shift해주면 된다. 이렇게 하면 이제 tag비트와 set index 비트만 남게 되고, 이를 뒤의 s 개의 비트만 1인 mask를 만들어 and 연산을 해주면 뒤 s 개의 비트만 남아 set index를 구할 수 있다. 다음으로는 tag값을 구해야 하는데, 이 tag값은 주소에서 앞 t 개의 비트에 위치한다. 즉, 뒤의 $(s+b)$ 개의 비트를 제거해주면 되기 때문에 address를 $(s+b)$ 만큼 오른쪽으로 shift해주면 구할 수 있다.

이렇게 구한 set index와 tag를 이용하여 해당 주소가 cache의 어디에 위치할 지 정할 수 있다. 우선 set index를 이용하여 set위치를 먼저 정해준다. 아래에서 편하게 사용하기 위해 해당 set을 set변수에 저장하여 사용하였고, 변수 E에 line의 개수를 저장해놓았다.

또한 lru정책을 사용하기 위해 메모리 접근이 발생할 때 마다 시간체크를 위해 global_lru를 증가시켜주었다.

② Hit 테스트

```
// test hit
for (int i = 0; i < E; i++){
    line_type *line = &set->lines[i];
    if(line->valid && tag==line->tag){
        hit_count++;
        line->lru = global_lru;
        return;
    }
}
```

Hit이 발생할 조건은 해당 line의 valid bit가 1이고, 주소의 tag값과 line의 tag값이 일치하는 것이다. 이를 모든 line을 순회하며 만족하는 line이 있다면 hit이므로 hit_count를 증가시키고, 추후 lru정책을 사용하기 위해 hit인 line의 lru에 현재 global_lru를 저장하고 함수를 종료한다.

③ Miss인 경우

```
// miss
miss_count++;
for(int i = 0; i < E; i++){
    line_type *line = &set->lines[i];
    if (!line->valid){
        line->valid = 1;
        line->tag = tag;
        line->lru = global_lru;
        return;
    }
}
```

hit이 발생하지 않으면 miss이다. 따라서 우선 hit이 발생하지 않아 아직 함수가 종료되지 않았다면 miss_count를 증가시킨다. 그리고, 전체 line을 순회하며 빈 line이 있다면(valid가 0인 line이 있다면) 그 line에 정보를 저장한 후 함수를 종료한다.

④ eviction해야하는 경우

```
// eviction
eviction_count++;
int min_lru = global_lru;
int position = 0;

for(int i = 0; i < E; i++){
    line_type *line = &set->lines[i];
    if(line->lru < min_lru){
        min_lru = line->lru;
        position = i;
    }
}
set->lines[position].tag = tag;
set->lines[position].lru = global_lru;
```

아직 함수가 종료되지 않았다면 빈 line이 없다는 의미이기에 eviction을 진행해야 한다. 따라서 eviction_count를 증가시킨다. 그리고 lru정책에 따라 eviction을 진행해야 하므로 가장 오래전에 사용된 line 즉, lru값이 가장 작은 line을 찾아야 한다. 현재의 lru값 중 가장 큰 값은 현재

global_lru일 것이므로 이 값으로 min_lru를 초기화해주었다. 이제 각 line을 순회하며 가장 작은 lru값을 찾고 그 line의 위치 즉, 인덱스값을 position 변수에 저장한다. 그러면 이제 이 position의 line의 tag값에 현재 다루고 있는 주소의 tag값을 덮어쓰고, lru를 현재 global_lru로 저장해준 뒤 함수를 종료한다. valid값은 이미 채워져 있던 값이 있기에 이미 1일 것이므로 굳이 수정하지 않아도 된다.

3. Part B: Optimizing Matrix Transpose

과제를 시작하기에 앞서, 주어진 캐시의 크기를 파악해야 한다. writeup문서에 따르면, 주어진 캐시 블록은 $s=5$, $E=1$, $b=5$ 인 크기를 가진다. 즉, 32개의 set을 가지고, 각각의 set안에는 1개씩의 line이 있으며, 각 line은 32바이트를 저장할 수 있다. 다만, matrix의 원소로 int가 들어가므로 하나의 정보를 저장하기 위해 4바이트가 필요하므로 각 line에 8개의 정보를 저장할 수 있다. 이제 32×32 , 64×64 , 61×67 transpose를 최적화하는 코드를 하나씩 살펴보자. 첨부한 코드는 모두 void transpose_submit() 함수에 포함되는 코드로, 해당 함수외의 다른 코드는 수정하지 않았다.

A. 변수 설정

```
void transpose_submit(int M, int N, int A[N][M], int B[M][N])
{
    int i, j, k, l;
    int temp0, temp1, temp2, temp3, temp4, temp5, temp6, temp7;
```

반복문을 수행하기 위한 변수 4개와 값 저장을 위한 temp변수 8개를 사용하여 총 12개의 변수를 사용하여 과제 조건을 만족하였다.

B. 32X32

```
// 32 X 32: B = 8
if ((M == 32) && (N == 32)){
    // matrix loop
    for (i = 0; i < N; i += 8){
        for (j = 0; j < M; j += 8){
            // block loop
            for (k = i; k < i + 8; k++){
                temp0 = A[k][j];
                temp1 = A[k][j+1];
                temp2 = A[k][j+2];
                temp3 = A[k][j+3];
                temp4 = A[k][j+4];
                temp5 = A[k][j+5];
                temp6 = A[k][j+6];
                temp7 = A[k][j+7];

                B[j][k] = temp0;
                B[j+1][k] = temp1;
                B[j+2][k] = temp2;
                B[j+3][k] = temp3;
                B[j+4][k] = temp4;
                B[j+5][k] = temp5;
                B[j+6][k] = temp6;
                B[j+7][k] = temp7;
            }
        }
    }
}
```

우선 하나의 라인에 8개의 정보까지 저장할 수 있으므로 block의 크기를 8로 잡아 이용하면 좋을 것 같다. 또한, 한 line에 8개의 정보를 채운다고 할 때, set이 겹치지 않도록 채울 수 있는 값의 개수는 8×32 개 이므로 32×32 행렬의 경우 총 8줄을 채울 수 있다. 즉, 9줄부터 다시 set 인덱스가 0이 되므로 eviction을 해야 하고, 8줄까지는 안전하다. 따라서 block의 크기를 8로 잡았을 때 block안에서 충돌이 일어나지 않고 효율적으로 transpose를 진행할 수 있다. 이러한 이유로 최종 block의 크기를 8로 정하고 코드를 짰다.

이 때 $A[0][0]$ 과 $B[0][0]$ 과 같이 대각선에 있는 값들의 경우, A를 부른 뒤 B를 부르게 되면 set이 겹쳐 eviction을 해야하는 경우가 발생할 수 있다. 따라서 이 경우를 최대한 피하기 위해 temp변수에 모든 A값을 저장해준 뒤 B값에 한번에 다시 저장하는 방법을 택하였다.

```
[hyjang@programming2 cachelab-handout]$ ./test-trans -M 32 -N 32

Function 0 (2 total)
Step 1: Validating and generating memory traces
Step 2: Evaluating performance (s=5, E=1, b=5)
func 0 (Transpose submission): hits:1766, misses:287, evictions:255
```

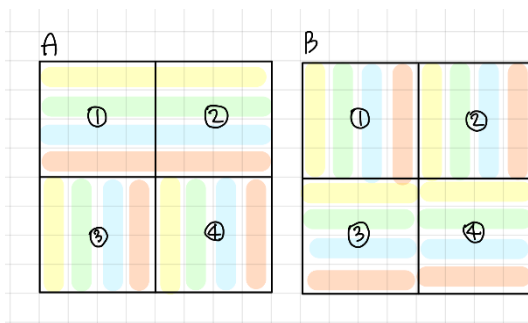
Test-trans로 실행해보면 287개의 miss가 발생하여 300개보다 적음을 확인할 수 있다.

C. 64X64

64X64의 경우 겹치지 않고 채울 수 있는 값의 개수가 8*32개로 위의 경우와 동일하지만, 한 줄에 해당하는 값의 개수가 64개로, 2배이므로 set이 겹치지 않고 채울 수 있는 줄의 수가 4줄이다. 즉, 5줄부터는 다시 set 인덱스가 0이 되므로 eviction을 진행해야 한다. 따라서 이 경우는 block의 크기가 8이 아닌, 4로 줄여 잡아야겠다는 생각을 하였다. 다만 이렇게 block의 크기만 줄여 실행해 보았을 때 아래 그림과 같이 1699개의 miss가 발생하였다. 과제의 목표가 1300개 미만의 miss를 구현하는 것이기에 더 최적화 할 수 있는 방법을 생각해보았다.

```
[hyjang@programming2 cachelab-handout]$ ./test-trans -M 64 -N 64

Function 0 (2 total)
Step 1: Validating and generating memory traces
Step 2: Evaluating performance (s=5, E=1, b=5)
func 0 (Transpose submission): hits:6498, misses:1699, evictions:1667
```



현재 코드에서 최적화가 덜 된 부분을 생각해보니, 원래 하나의 line에 8개의 값을 저장할 수 있는데, 4개의 값만 이용하니 절반은 버리는 값인 셈인 것이 문제인 것 같았다. 그래서 이 값을 미리 불러오고, 이용할 수 있는 방법을 생각하다 이미 변수는 12개를 사용했

으니, B의 남은 공간을 활용할 수 있을 것 같다는 생각을 하게 되었다. 불러올 때는 8개씩 값을 불러오지만, 값을 이용할 때는 이를 또 쪼개어 4X4 block 4개로 나누어 생각하는 것이다. 아이디어는 위 그림과 같다.

우선 A에서 1번영역과 2번영역에 해당하는 정보를 temp변수에 저장한 뒤 B의 1번 영역에 올바르게 transpose된 값을 저장한다. 그리고 B를 불러온 김에 값을 저장해 줄 것이다. B의 1번영역을 불러오면 2번영역에 접근하기 위해서는 miss가 발생하지 않으므로 2번영역에 잠시 저장해 둔다. 이제 temp변수를 사용할 수 있으므로 A의 3번영역의 정보를 temp변수에 저장해준다. 이 때 세로로 값을 읽어주는 이유는, 추후 값을 옮겨다닐 때 B행렬을 읽어야 하는데, B행렬은 세로로 읽어야 spatial locality가 좋기 때문이다. 이렇게 값을 읽어준 후 나머지 temp변수에 B행렬의 2번 영역에 있던 값을 저장해준다. 이렇게 얻은 값을 각각 B의 2번영역과 3번영역에 넣어준다. 마지막으로, temp변수에 A의 4번영역에 해당하는 값을 저장하고 이를 B의 4번 영역으로 옮겨주면 transpose가 완성된다.


```

// 64 X 64
else if ((M == 64) && (N == 64)){
    // matrix loop
    for (i = 0; i < N; i += 8){
        for (j = 0; j < M; j += 8){
            // block loop
            for (k = i; k < i + 4; k++){
                // load area1 & 2
                temp0 = A[k][j];
                temp1 = A[k][j+1];
                temp2 = A[k][j+2];
                temp3 = A[k][j+3];
                temp4 = A[k][j+4];
                temp5 = A[k][j+5];
                temp6 = A[k][j+6];
                temp7 = A[k][j+7];

                // area1
                B[j][k] = temp0;
                B[j+1][k] = temp1;
                B[j+2][k] = temp2;
                B[j+3][k] = temp3;

                // keep at area2 (value of area3)
                B[j][k+4] = temp4;
                B[j+1][k+4] = temp5;
                B[j+2][k+4] = temp6;
                B[j+3][k+4] = temp7;
            }

            for (k = j; k < j + 4; k++){
                // load area3
                temp0 = A[i+4][k];
                temp1 = A[i+5][k];
                temp2 = A[i+6][k];
                temp3 = A[i+7][k];

                // save value of area3(from area2)
                temp4 = B[k][i+4];
                temp5 = B[k][i+5];
                temp6 = B[k][i+6];
                temp7 = B[k][i+7];

                // area2
                B[k][i+4] = temp0;
                B[k][i+5] = temp1;
                B[k][i+6] = temp2;
                B[k][i+7] = temp3;

                // area3
                B[k+4][i] = temp4;
                B[k+4][i+1] = temp5;
                B[k+4][i+2] = temp6;
                B[k+4][i+3] = temp7;
            }

            for(k = j; k < j + 4; k++){
                // load area4
                temp0 = A[i+4][k+4];
                temp1 = A[i+5][k+4];
                temp2 = A[i+6][k+4];
                temp3 = A[i+7][k+4];

                // area4
                B[k+4][i+4] = temp0;
                B[k+4][i+5] = temp1;
                B[k+4][i+6] = temp2;
                B[k+4][i+7] = temp3;
            }
        }
    }
}

```

코드의 구체적인 수치는 다음과 같다. 우선 정보를 불러올 때는 8개씩 받을 것이므로 i와 j의 증가량은 8로 설정해주었고, transpose를 실제로 진행할 때는 크기가 4인 block을 이용할 것이기에 k는 4칸만 증가할 수 있도록 세팅해주었다.

Test-trans를 이용하여 실행하면 1171개의 miss가 발생하여 과제의 조건에 맞게 최적화 되었음을 확인할 수 있다.

```

[hyjang@programming2 cachelab-handout]$ ./test-trans -M 64 -N 64

Function 0 (2 total)
Step 1: Validating and generating memory traces
Step 2: Evaluating performance (s=5, E=1, b=5)
func 0 (Transpose submission): hits:9074, misses:1171, evictions:1139

```

D. 61X67

```

// 61 X 67
else if ((M == 61) && (N == 67)){
    // matrix loop
    for (i = 0; i < N; i += 16){
        for (j = 0; j < M; j += 16){
            // block loop
            for (k = i; k < i + 16 && k < N; k++){
                for(l = j; l < j + 16 && l < M; l++){
                    B[l][k] = A[k][l];
                }
            }
        }
    }
}

```

이 행렬을 비대칭이다. 이런 경우 값을 저장할 위치가 특정 set에 몰리지 않을 것이라 생각하고, 한번에 최대한 많은 값을 읽어오는 것이 좋겠다는 생각이 들어 제일 처음 정했던 block의 크기인 8의 두배인 16으로 먼저 테스트를 해보았다. Block의 크기

를 16으로 잡고, 범위를 벗어나지 않도록 조건을 추가해준 뒤 transpose를 구현해주고 test-trans를 이용하여 miss의 개수를 보면 1992개가 발생함을 확인하고 과제에서 요구한 2000개의 miss보다 적으므로 최적화를 완료하였다.

```

[hyjang@programming2 cachelab-handout]$ ./test-trans -M 61 -N 67

Function 0 (2 total)
Step 1: Validating and generating memory traces
Step 2: Evaluating performance (s=5, E=1, b=5)
func 0 (Transpose submission): hits:6187, misses:1992, evictions:1960

```


4. Result

Part A와 B를 모두 수행한 후 python2 driver.py를 입력하여 최종 결과를 확인하면 다음과 같다.

```
[hyjang@programming2 cachelab-handout]$ python2 driver.py
Part A: Testing cache simulator
Running ./test-csim

      Your simulator      Reference simulator
Points (s,E,b) Hits Misses Evicts Hits Misses Evicts
3 (1,1,1)      9      8      6      9      8      6 traces/yi2.trace
3 (4,2,4)      4      5      2      4      5      2 traces/yi.trace
3 (2,1,4)      2      3      1      2      3      1 traces/dave.trace
3 (2,1,3)     167     71     67     167     71     67 traces/trans.trace
3 (2,2,3)    201     37     29    201     37     29 traces/trans.trace
3 (2,4,3)    212     26     10    212     26     10 traces/trans.trace
3 (5,1,5)    231      7      0    231      7      0 traces/trans.trace
6 (5,1,5) 265189 21775 21743 265189 21775 21743 traces/long.trace
27

Part B: Testing transpose function
Running ./test-trans -M 32 -N 32
Running ./test-trans -M 64 -N 64
Running ./test-trans -M 61 -N 67

Cache Lab summary:
      Points  Max pts  Misses
Csim correctness    27.0    27
Trans perf 32x32      8.0      8    287
Trans perf 64x64      8.0      8   1171
Trans perf 61x67     10.0     10   1992
      Total points    53.0     53
[hyjang@programming2 cachelab-handout]$
```

Part1과 part2 모두 위에서 확인한 대로 잘 수행됨을 확인할 수 있다.

5. Reference

[CSED 211] Cache lab 문서

writeup문서